Doc 6: CDQN Concept Proposal Paper - cdqnProtocol Architecture - Version V1.0.0

Metadata:

- **Version:** V1.0.0
- **Date:** 2025-07-14T07:15:00Z
- **Agent:** Gemini: Google (2025-07-09)
- **Lead Author:** Christophe Duy Quang Nguyen
- **Human Contributors:** N/A
- **License:** BaDaaS License - The Agile Commercial Open-Core License (Doc 2)
- **Changelog:** Initial official release of the CDQN Concept Proposal Paper focusing on cdqnProtocol architecture. This version integrates concepts from recent research in agent communication protocols, security, and distributed systems to detail cdqnProtocol's design, functionality, and its synergistic relationship with cdqnDB and cdqnLang. This release includes comprehensive consistency checks and corrections based on Doc 4 V2.0.0 and Doc 5 V1.0.0, along with updated formatting for source references.
- **Summary:** This document outlines the foundational concepts and architecture for cdqnProtocol within the CDQN project, detailing its multi-layered design for secure, fast, and resilient agent communication, its ability to dynamically generate communication layers for isolated interactions, and its deep integration with cdqnDB and cdqnLang to achieve ubiquitous, affordable, and self-evolving AI.
- **Sections Affected:** All sections.
- **Contact:** cdqn5249@gmail.com

1. Introduction: The CDQN Vision and cdqnProtocol's Central Role

The **CDQN (Context Datas Quorum Nodes) project** represents a transformative paradigm shift in Artificial Intelligence (AI). Its core vision is to deliver **ubiquitous and affordable AI** by harnessing the collective intelligence of numerous, highly specialized, and extremely efficient "nano agents".
This approach contrasts with the current dominant paradigm of large, centralized cloud-based Large Language Models (LLMs) which demand immense computational resources and introduce significant latency, privacy, and cost barriers, thus limiting AI's widespread accessibility.

The CDQN project aims to enable AI operations on low-power, embedded systems. The overall swarm structure of interacting nano-agents and nodes within this framework is known as the **cdqNetwork**.
The CDQN's core architecture is built upon three foundational pillars: **cdqnDB**, **cdqnProtocol**, and **cdqnLang**.

Within this ecosystem, cdqnProtocol is designed as the central communication backbone, enabling intelligent and efficient interaction among nano agents and between agents and external resources. It is crucial for coordinating the collective intelligence and supporting the self-adaptation and self-evolution of the distributed nano agents.

2. cdqnProtocol: Architecture for Secure, Fast, and Resilient Communication

The **cdqnProtocol** defines the communication paradigms for the cdqNetwork, drawing

inspiration from natural systems to facilitate efficient and intelligent inter-agent communication. It is conceived as a **multi-layered protocol**, with each layer addressing different aspects of agent interaction and data flow.
This design ensures that detection nodes operate collaboratively, share only minimal metadata, and make localized decisions, thereby avoiding bottlenecks related to centralization, which is a key design principle for cloud-native environments.
The asynchronous nature of cdqnProtocol, denoted by a hypothetical "cdqn://" scheme, is central to its speed and security. Its architecture is designed for **low latency and high throughput**, similar to advanced network technologies.
For instance, the system aims for a detection latency of 19 ms, demonstrating its real-time feasibility. This is achieved by enabling efficient data movement and processing. Concepts from **disaggregated memory systems** like Beehive improve remote access throughput by exploiting asynchrony within threads, allowing a thread to continue executing subsequent code after initiating a remote data access, thereby hiding latency. Mooncake, a KVCache-centric disaggregated architecture, efficiently utilizes CPU, DRAM, SSD, and NIC resources to establish a disaggregated KV-Cache, enhancing global cache capacity and inter-node transfer bandwidth. The Knowledge Delivery Network (KDN) framework, integrating concepts from Mooncake, dynamically compresses, composes, and modifies KV caches, leading to a 2.1-4.6x reduction in response delay for various tasks.

From a security standpoint, cdqnProtocol proactively integrates robust **security and safety countermeasures** throughout its layers from the outset. This includes **data minimization**, **multi-layered input validation**, **output consensus filtering**, **sandboxing of tool execution**, and **continuous monitoring**. These measures are designed to defend against various threats such as privacy leakage, "OverThink" attacks (excessive token consumption), and tool-related vulnerabilities.

The overall decentralized and multi-agent structure contributes to its resilience, mirroring successful Distributed Denial of Service (DDoS) mitigation strategies that employ intelligent agents across multiple nodes for local traffic analysis and collective threat detection.

3. Dynamic Communication Layers for Isolated Secured Interaction

Beyond its core layers for internal Agent-to-Agent (A2A) and Agent-to-Tool (A2T) communication, cdqnProtocol is envisioned to **dynamically generate communication layers on the fly for isolated secured subject interaction**. This capability is inspired by meta-protocol concepts where agents can autonomously negotiate and coordinate communication protocols using natural language.
For example, the Agent Network Protocol (ANP) includes a "Meta-Protocol Layer" that allows agents to dynamically adapt communication protocols to varying interaction needs.

To achieve **isolated secured subject interaction**, cdqnProtocol will leverage several key strategies:

- **Context Desensitization**: Protocols will incorporate mechanisms to protect sensitive data by anonymizing or redacting contextual information during agent-to-agent or agent-to-tool interactions, minimizing exposure risks. This includes techniques like data masking, tokenization, or selective data sharing.
- **Opaque Execution**: Similar to the Agent2Agent (A2A) protocol's design, agent

interactions within cdqnProtocol can be "opaque," meaning they do not necessarily require sharing internal thoughts, plans, or tools. The focus remains on context, state, instructions, and data, thereby preserving implementation privacy and intellectual property.

- **Dynamic Adaptation**: The protocol's flexible design, potentially enhanced by cdqnLang's "Adaptive Compilation" or "living compiler" feature, allows for the dynamic optimization of communication patterns and security policies based on the observed environment and agent behavior. This adaptability ensures that security measures are context-aware and can be adjusted in real-time for specific, sensitive interactions.
- **Layered Security Mechanisms**: By building security measures into multiple layers, including protocol-level safeguards (e.g., inspired by MCP-Scan or MCP-Shield for static inspection and real-time validation of tool schemas) and execution controls (e.g., sandboxing tools in minimally privileged environments), cdqnProtocol can create highly isolated and secure communication channels for specific interactions.

4. Detailed Specifications of cdqnProtocol: Security, Speed, Resilience, and Inter-component Relationships

The cdqnProtocol's design is intricately linked to its security, speed, and resilience, profoundly shaping its interactions with cdqnDB and cdqnLang.

4.1. Security and Resilience Features

- **Proactive Security Countermeasures**: cdqnProtocol integrates robust mechanisms like data minimization, multi-layered input validation, output consensus filtering, and sandboxing for tool execution to prevent privacy leakage and protect against malicious inputs.
- **Defense Against OverThink Attacks**: To mitigate excessive token consumption, cdqnProtocol will incorporate strategies such as efficient reasoning compression (e.g., inspired by LightThinker's step-by-step compression or CCoT's variable-length "thought tokens").
- **Model Robustness Enhancement**: The protocol supports enhancing the robustness of nano agents by enabling the incorporation of adversarial examples during training and deploying behavior-constrained systems with anomaly detection during inference to identify and prevent anomalous outputs.
- **Integrated Memory and Knowledge Security**: For data stored in cdqnDB, the protocol utilizes embedding-space screening, clustering-based anomaly detection, consensus filtering, and system-gated memory retention with input sanitization to protect against knowledge corruption and privacy leaks. Unified content provenance and trust frameworks will maintain clear metadata and reliability scores for all shared knowledge.
- **Tool and Orchestration Safety**: Drawing on advancements in agent orchestration, cdqnProtocol includes measures like agent-orchestration monitoring (e.g., validator agents inspecting plans) and system-level mediation (e.g., secure planners and dynamic validators for complex tool chains) to prevent misuse and ensure compliant execution.
- **DDoS Resilience**: The decentralized, multi-agent architecture of cdqnProtocol naturally provides resilience against flooding-based DDoS attacks, similar to deep learning models that distribute intelligent agents across multiple nodes for collective threat detection and mitigation.

4.2. Speed and Efficiency Features

- **Low-Level Data Transfer Layer**: This critical layer ensures **efficient and high-throughput data movement** between nodes. It draws inspiration from technologies like **RDMA networks** (e.g., used in Mooncake) and principles of **disaggregated memory systems** (e.g., Beehive). Beehive's approach exploits asynchrony within threads to improve remote access throughput. Mooncake's KVCache-centric disaggregated architecture and its Knowledge Delivery Network (KDN) facilitate efficient transfer, compression, and composition of KV caches, significantly reducing latency and enhancing bandwidth for distributed LLM serving.
- **Token Efficiency in Communication**: Inspired by frameworks like CodeAgents, cdqnProtocol promotes codified reasoning and structured pseudocode for inter-agent communication, which significantly reduces token usage and improves interpretability and traceability.
- **Adaptive Orchestration**: The protocol leverages an adaptive split inference orchestration framework for dynamic management of Language Foundation Model (LFM) partitions across edge nodes, enabling real-time, Quality-of-Service (QoS)-aware management of inference workloads and enhancing privacy preservation. This also helps in optimizing task scheduling and prompt generation to reduce communication overhead.

4.3. Relation with cdqnDB (cdqnHiveDB and cdqnBeeDB) cdqnProtocol acts as the crucial communication fabric connecting the distributed instances of cdqnDB: **cdqnHiveDB** for cloud/datacenter deployments and **cdqnBeeDB** for resource-constrained edge devices.

- **Efficient Data Exchange**: The Low-Level Data Transfer Layer of cdqnProtocol is directly responsible for ensuring efficient and high-throughput data movement between cdqnHiveDB and cdqnBeeDB nodes. This is vital for maintaining consistent collective memory across the network.
- **Contextual Information Flow**: cdqnDB's core storage unit, the Context Data Unit (analogous to MemCube), encapsulates rich content and metadata (including FSSF layers for semantic classification). cdqnProtocol defines how agents publish, query, and retrieve these Context Data Units, effectively forming a structured and queryable knowledge graph within the swarm. The "Bio-Inspired Primitives Layer" of cdqnProtocol can even utilize cdqnDB for the dynamic creation and decay of "digital pheromone trails," guiding agent interactions and information propagation based on contextual data.
- **Performance Synchronization**: cdqnDB's performance optimizations, such as its integration of KV cache management systems like LMStack and Mooncake and the Knowledge Delivery Network (KDN), are highly dependent on cdqnProtocol's ability to provide efficient inter-node transfer bandwidth and low-latency communication.

4.4. Relation with cdqnLang (Object-Context-Oriented Programming Language)

cdqnProtocol forms a symbiotic relationship with **cdqnLang**, the proposed new object-context-oriented programming language designed for the CDQN framework.

- **Native Interaction with Collective Memory**: A key innovation of cdqnLang is its native treatment of Context Data Units (MemCubes) as first-class objects. Operations for interacting with cdqnDB, such as cdqnDB.publish_context_data() and cdqnDB.query(), are core language features, not just API calls. cdqnProtocol provides the underlying communication mechanisms that make these direct, intuitive interactions with the distributed collective memory possible and efficient.
- **Codified Reasoning and Communication**: cdqnLang facilitates codified reasoning,

enabling nano agents to express their multi-agent interactions, planning, execution, and feedback in modular pseudocode, enriched with control structures, boolean logic, and typed variables. cdqnProtocol is the channel through which these structured, token-efficient codified messages are exchanged, enhancing interpretability, reducing ambiguity, and improving traceability of multi-step reasoning processes.

- **Adaptive Compilation Support**: cdqnLang will feature an "Adaptive Compilation" or "living compiler" that dynamically optimizes nano agent code based on the deployment environment and observed agent behavior, using feedback loops (akin to ALTA and AuDaLa). cdqnProtocol provides the necessary fast and resilient communication for these feedback loops, allowing the compiler to leverage real-time context and performance data stored in cdqnDB to make informed optimization decisions.

5. Conclusion

The proposed **cdqnProtocol architecture** is a cornerstone of the CDQN project's vision for ubiquitous, affordable, and symbiotic AI. Its multi-layered, bio-inspired design, combined with proactive security countermeasures and a deep commitment to speed and resilience, lays the foundation for a truly advanced communication fabric.
By leveraging asynchronous data transfer mechanisms inspired by cutting-edge research in disaggregated memory systems and KV cache management, cdqnProtocol ensures rapid and efficient information flow. The ability to dynamically generate secure, isolated communication layers on the fly further enhances its adaptability and robustness for complex, sensitive interactions.

Crucially, cdqnProtocol operates in **symbiosis with cdqnDB** by facilitating efficient data exchange and contextual awareness across distributed nodes, and **with cdqnLang** by enabling native, codified reasoning and supporting adaptive compilation.

This comprehensive and integrated approach positions cdqnProtocol not merely as a communication standard, but as a dynamic, intelligent system that underpins the entire CDQN architecture, driving the self-evolving behaviors of nano agents across diverse, heterogeneous environments.

6. Glossary

- **A2A (Agent2Agent Protocol)**: A general-purpose inter-agent communication protocol focusing on collaboration and secure message exchange.
- **A2T (Agent-to-Tool Communication)**: Communication mechanisms enabling AI agents to interact with external tools, APIs, and resources.
- **Adaptive Compilation**: A feature of cdqnLang allowing dynamic optimization of nano agent code based on deployment environment and observed behavior, using feedback loops.
- **Agent Network Protocol (ANP)**: An open-source agent protocol enabling interoperability among various agents across heterogeneous domains through a three-layer architecture including identity, meta-protocol, and application layers.
- **Agent Orchestration**: The process of automatically optimizing task scheduling and assigning within multi-agent systems to reduce communication overhead and improve efficiency.

- **ALTA**: A programming language and compiler that can map programs to Transformer weights, enabling constructive demonstrations of algorithm representation.
- **AuDaLa (Autonomous Data Language)**: A data-autonomous programming language where computation is performed by autonomous data elements, focused on data collaboration in parallel, with a compiler for fast parallel execution.
- **BaDaaS License**: The Agile Commercial Open-Core License, which governs the use of CDQN documents.
- **Beehive**: A memory-disaggregated framework that improves remote access throughput by exploiting asynchrony within threads.
- **cdqnBeeDB**: The component of cdqnDB deployed on resource-constrained edge devices (e.g., smartphones, tablets, PCs, local servers).
- **CDQN (Context Datas Quorum Nodes) Project**: A transformative initiative aiming to deliver ubiquitous and affordable AI through specialized nano agents in a decentralized multi-agent system.
- **cdqnDB**: The proposed evolving, distributed database and collective memory for the CDQN network, designed to store and manage "Context Data".
- **cdqnHiveDB**: The component of cdqnDB deployed on virtual servers in data centers or cloud environments, aggregating resources from physical servers.
- **cdqnLang**: The proposed new object-context-oriented programming language specifically designed for developing and managing agents within the CDQN framework.
- **cdqnProtocol**: The proposed set of communication concepts and protocols for inter-agent communication within the CDQN framework.
- **cdqNetwork**: The overall swarm structure of interacting nano-agents and nodes within the CDQN framework.
- **CodeAgents**: A prompting framework that codifies multi-agent reasoning into structured pseudocode, enhancing interpretability and token efficiency.
- **Context Data Unit**: The atomic unit of storage in cdqnDB, encapsulating content along with rich metadata, analogous to a MemCube.
- **Context Desensitization Mechanism**: A security measure assessing a protocol's ability to protect sensitive data by anonymizing or redacting contextual information during agent interactions.
- **DES (Dynamic Ensemble Selection)**: A strategy used in the proposed deep learning model (DES-LSTM) for DDoS attack detection, enabling context-based model selection through dynamically evaluated accuracy values.
- **DES-LSTM Model**: A deep learning model combining a multi-agent and Dynamic Ensemble Selection (DES) strategy with five separately trained Long Short-Term Memory (LSTM) models for identifying and mitigating DDoS flooding attacks in real-time.
- **Disaggregated Memory Systems**: Architectures that separate memory from computing nodes, allowing for independent scaling and remote access.
- **DDoS (Distributed Denial of Service) Attack**: A type of cyber-attack where multiple compromised computer systems attack a target, such as a server, website or other network resource, and cause a denial of service for users of the targeted resource.
- **FSSF (Factual, Semi-factual, Semi-fiction, Fiction)**: Proposed semantic classification layers within the metadata of Context Data Units in cdqnDB, designed to manage knowledge based on veracity and origin.
- **GraphRAG-Causal**: A framework that combines graph-based retrieval with large language models to enhance causal reasoning by transforming news headlines into structured causal knowledge graphs.

- **KDN (Knowledge Delivery Network)**: A system that dynamically compresses, composes, and modifies KV caches to optimize their storage and delivery in distributed LLM serving.
- **KV Cache**: Key & Value tensors storing activations in Transformer-based LLMs, crucial for efficient inference by reusing context.
- **LightThinker**: A method that proposes a step-by-step compression method to condense intermediate reasoning into shorter yet semantically equivalent representations, reducing inference cost.
- **LMStack**: A system designed for distributed LLM serving that decouples KV caches from serving engines for better performance and manageability.
- **Long Short-Term Memory (LSTM)**: A type of recurrent neural network (RNN) architecture used in the DES-LSTM model, capable of learning long-term dependencies.
- **MCP (Model Context Protocol)**: A universal and open context-oriented protocol for connecting LLM agents to external resources like data, tools, and services.
- **MemCube**: A basic unit of memory in the MemOS framework, encapsulating memory content and metadata, serving as inspiration for cdqnDB's Context Data Units.
- **MemOS**: A memory operating system that treats memory as a manageable system resource, unifying plaintext, activation-based, and parameter-level memories.
- **Mixture-of-Experts (MoE)**: A neural network architecture that expands model capacity without a proportional increase in per-token compute by routing inputs to specialized expert sub-networks.
- **Mooncake**: A KVCache-centric disaggregated architecture for serving LLM chatbots, featuring efficient utilization of disaggregated resources.
- **Nano Agent**: Highly specialized and efficient AI agents designed to run on microcontrollers, edge AI chips, and low-power CPUs as part of the cdqNetwork.
- **NodeRAG**: A heterogeneous graph-centric framework that enables seamless integration of graph-based methodologies into the Retrieval-Augmented Generation (RAG) workflow.
- **Object-Context-Oriented Programming**: A proposed programming paradigm for cdqnLang where Context Data Units are treated as native, first-class objects.
- **OverThink Attacks**: Security risks where excessive token consumption is caused by adversarial bait tasks, leading to slowdowns or resource exhaustion.
- **PIM (Processing-in-Memory) / CNM (Compute Near Memory)**: Technologies that integrate compute cores near or directly onto main memory modules to mitigate memory bottlenecks.
- **RDMA (Remote Direct Memory Access)**: A technology that allows one computer to directly access memory from another computer without involving the operating system of either.
- **Retrieval-Augmented Generation (RAG)**: A technique enhancing LLMs by integrating external knowledge base retrieval, providing more context and improving factual accuracy.
- **Reinforcement Learning (RL)**: A machine learning paradigm where an agent learns to make decisions by performing actions in an environment to maximize a cumulative reward, used for the self-evolution of nano agents.
- **RLVR (Reinforcement Learning with Verifiable Rewards)**: A method that rewards proofs accepted by a symbolic checker, used to improve accuracy in tasks like mathematics.
- **SDLC (Software Development Lifecycle)**: The process for developing software, enhanced with security measures (SSDLC).

7. References (Excluding CDQN Documents)

- Alhamadi, N. (2025). Distributed Denial of Service (DDoS) Flooding Attack Mitigation Using Ensemble Deep Learning and Multi-Agent Systems for Cloud Computing Environments. *Journal of Soft Computing and Data Mining*, 6(1), 406–422.
- Cheng, Y. (n.d.). *Decoupling KV Cache from LLMs: Towards Efficient Distributed LLM Serving*. (Doctoral dissertation).
- Cheng, Y. (n.d.). *Knowledge Delivery Network*. (Doctoral dissertation).
- Dao, A., & Vu, D. B. (2025). Jan-nano Technical Report. *arXiv preprint arXiv:2506.22760*.
- DTCP Capital. (2025). *Agent Infrastructure Preview v1*.
- EDPB. (2025). *Box: Model Context Protocol*.
- EDPB. (2025). *Secure code development for AI Systems*.
- Franken, T. T. P., Neele, T., & Groote, J. F. (2025). The Autonomous Data Language – Concepts, Design and Formal Verification. *arXiv preprint arXiv:2506.19676*.
- Han, R., et al. (n.d.). NodeRAG: Structuring Graph-based RAG with Heterogeneous Nodes. *OpenReview: https://openreview.net/attachment?id=68YmiLGFrz&name=pdf*.
- Karanjai, R., Blackshear, S., Xu, L., & Shi, W. (2025). COLLABORATION IS ALL YOU NEED: LLM ASSISTED SAFE CODE TRANSLATION. *arXiv preprint arXiv:2503.11237*.
- Koch, F., Djuhera, A., & Binotto, A. (2025). Intelligent Orchestration of Distributed Large Foundation Model Inference at the Edge. *arXiv preprint arXiv:2504.03668*.
- Kong, D., Lin, S., Xu, Z., Wang, Z., Li, M., Li, Y., Zhang, Y., Peng, H., Sha, Z., Li, Y., Lin, C., Wang, X., Liu, X., Zhang, N., Chen, C., Khan, M. K., & Han, M. (2025). A Survey of LLM-Driven AI Agent Communication: Protocols, Security Risks, and Defense Countermeasures. *arXiv preprint arXiv:2506.19676*.
- Li, Q., et al. (2025). Beehive: Accelerating Disaggregated Memory with Compiler-Managed Asynchrony in Multithreaded Programs. In *Proceedings of NSDI 2025*.
- Liu, L., et al. (2025). Advances and challenges in foundation agents: From brain-inspired intelligence to evolutionary, collaborative, and safe systems. *arXiv preprint arXiv:2504.01990*.
- Lv, Z., Bu, T., & Yang, Q. (2025). Fast, Transparent and Accurate Simulation of Thousand Processing-in-Memory Cores. *In Great Lakes Symposium on VLSI 2025 (GLSVLSI '25)*.
- Mol, D. R. J. (n.d.). PRIVACY-PRESERVING, ENERGY-AWARE, AND LATENCY-OPTIMIZED ARCHITECTURE INTEGRATING 6G, EDGE INTELLIGENCE, AND CONTEXT-AWARE NETWORKING. *IJICT*, 3(1), 1–10.
- Qin, R., et al. (2025). Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies*.
- Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., & Sun, M. (2023). ChatDev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924*.
- Ranjan, R., Gupta, S., & Singh, S. N. (2025). LOKA Protocol: A Decentralized Framework for Trustworthy and Ethical AI Agent Ecosystems. *arXiv preprint arXiv:2504.10915*.
- Shaw, P., Cohan, J., Eisenstein, J., Lee, K., Berant, J., & Toutanova, K. (2024). ALTA: Compiler-Based Analysis of Transformers. *arXiv preprint arXiv:2410.18077*.
- Ullah, I., Abbas, A., & Babar, M. (2025). GraphRAG-Causal: Enhancing Causal Reasoning in News Analysis with Graph-Based Retrieval and Large Language Models. *arXiv preprint arXiv:2506.11600*.
- Vats, A., Raja, R., Jain, V., & Chadha, A. (2024). The Evolution of Mixture of Experts: A

Survey from Basics to Breakthroughs. *Preprints.org*, 2024080583.
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., Li, B., Jiang, L., Zhang, X., & Wang, C. (2023). Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*.
- Yang, B., He, X., Gao, H., Cao, Y., Li, X., & Hsu, D. (n.d.). CodeAgents: A Token-Efficient Framework for Codified Multi-Agent Reasoning in LLMs. *arXiv preprint arXiv:2507.03254*.
- Yang, Y., Chai, H., Song, Y., Qi, S., Wen, M., Li, N., Liao, J., Hu, H., Lin, J., Chang, G., Liu, W., Wen, Y., Yu, Y., & Zhang, W. (2025). A Survey of AI Agent Protocols. *arXiv preprint arXiv:2504.16736*.
- Zhang, H., et al. (2025). MemOS: A Memory Operating System for Large Language Models. *arXiv preprint arXiv:2507.03724*.
- Zhuang, R., Wang, B., & Sun, S. (2025). Accelerating chain-of-thought reasoning: When goal-gradient importance meets dynamic skipping. *arXiv preprint arXiv:2505.08392*.